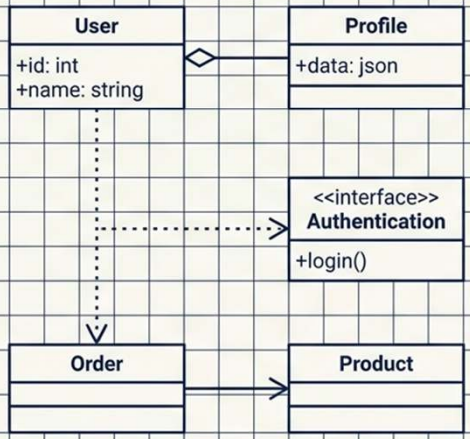
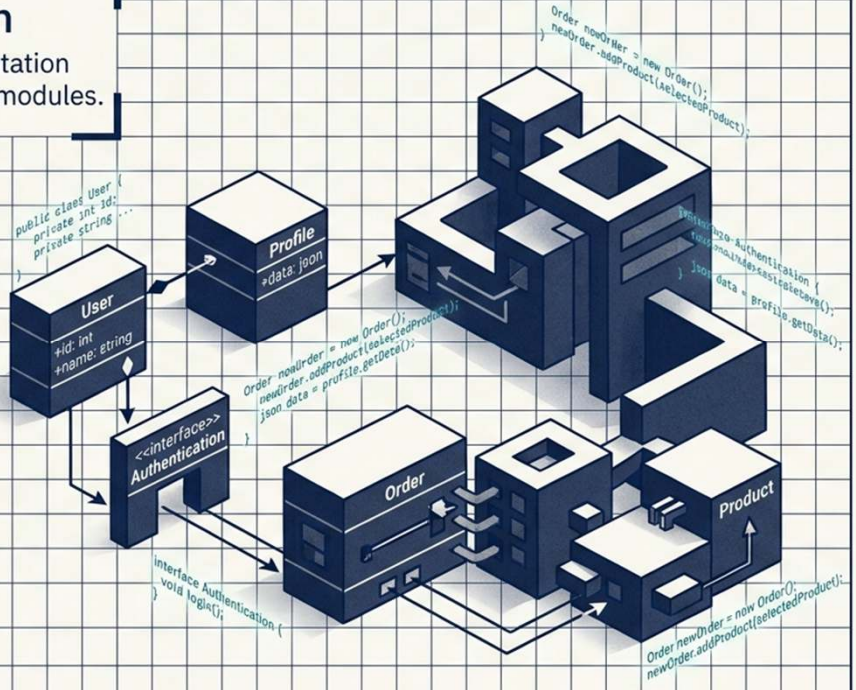
Software Architecture Design

Assignment 2 Playbook: Patterns & Implementation
Evolving from global blueprints to executable modules.





OBJECTIVE

Extend the Assignment 1 structural foundation into an implementation-ready architecture.

CONTEXT

Applied to the existing real-world case study established by the group.

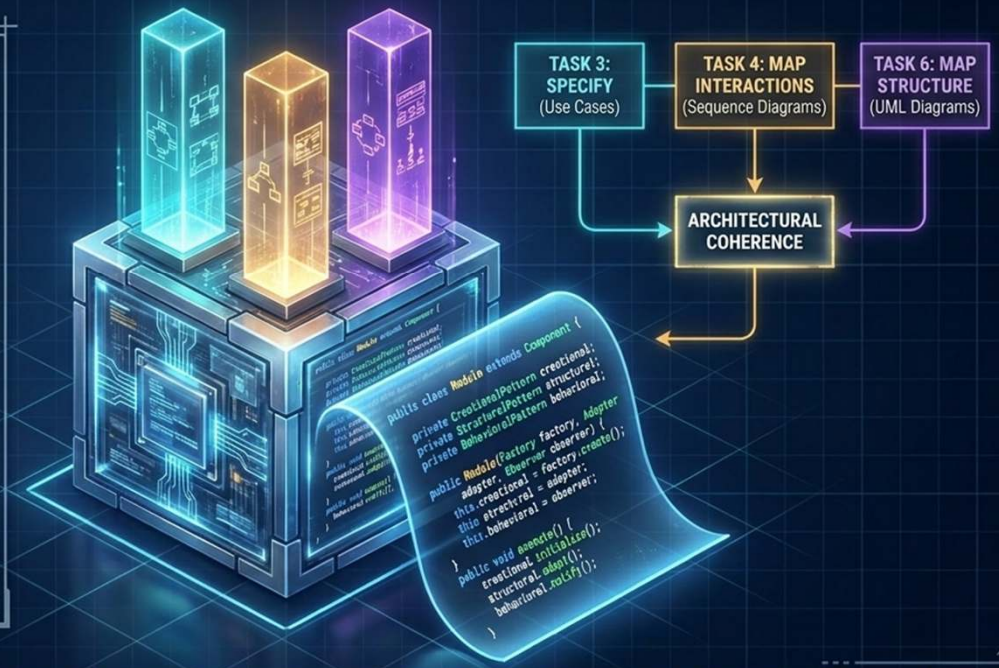
CORE CHALLENGE

Inject standardized design patterns and generate concrete code samples while maintaining strict architectural coherence.

The Assignment 2 Mandate: Evolving the Individual Module.

For every assigned module, the owner must design, specify, and align:

1. One Creational Design Pattern
2. One Structural Design Pattern
3. One Behavioral Design Pattern
4. Code samples generated directly from the architectural diagrams.



Synthesis Matrix: The Pattern Triad

Creational Pattern	Structural Pattern	Behavioral Pattern
Object Instantiation	Object Composition	Object Communication
How do we efficiently create this?	How do we assemble and fit this?	How do these entities talk?
Optimizes memory footprint and setup overhead.	Ensures scalability and seamless integration with the global A1 architecture.	Manages complex state changes and algorithmic flow.

Designing the Creational Pattern



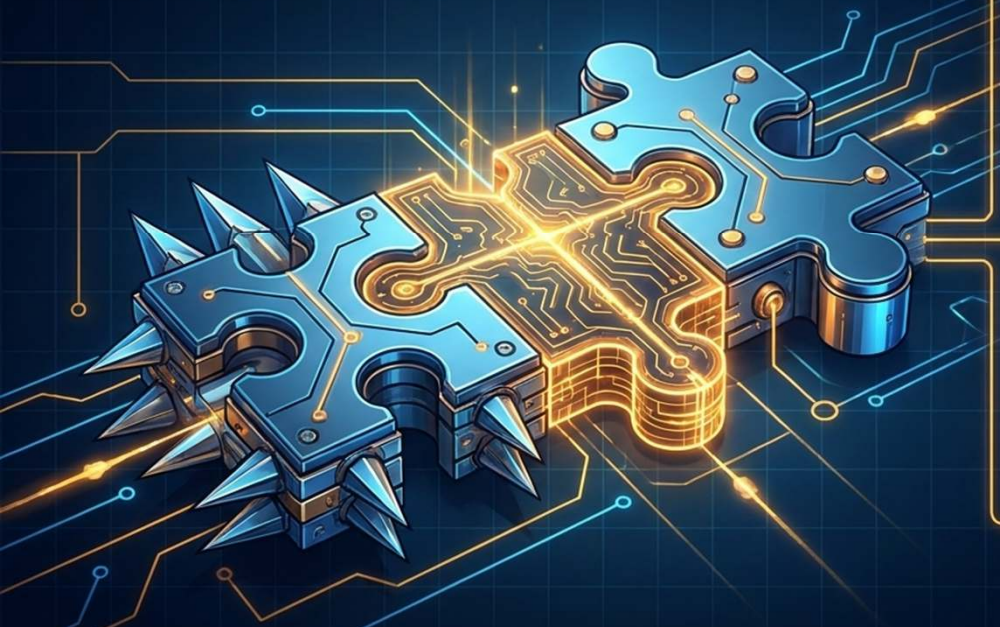
Core Principle

Hide the creation logic rather than instantiating objects directly.

Examples

Singleton, Factory Method, Abstract Factory, Builder.

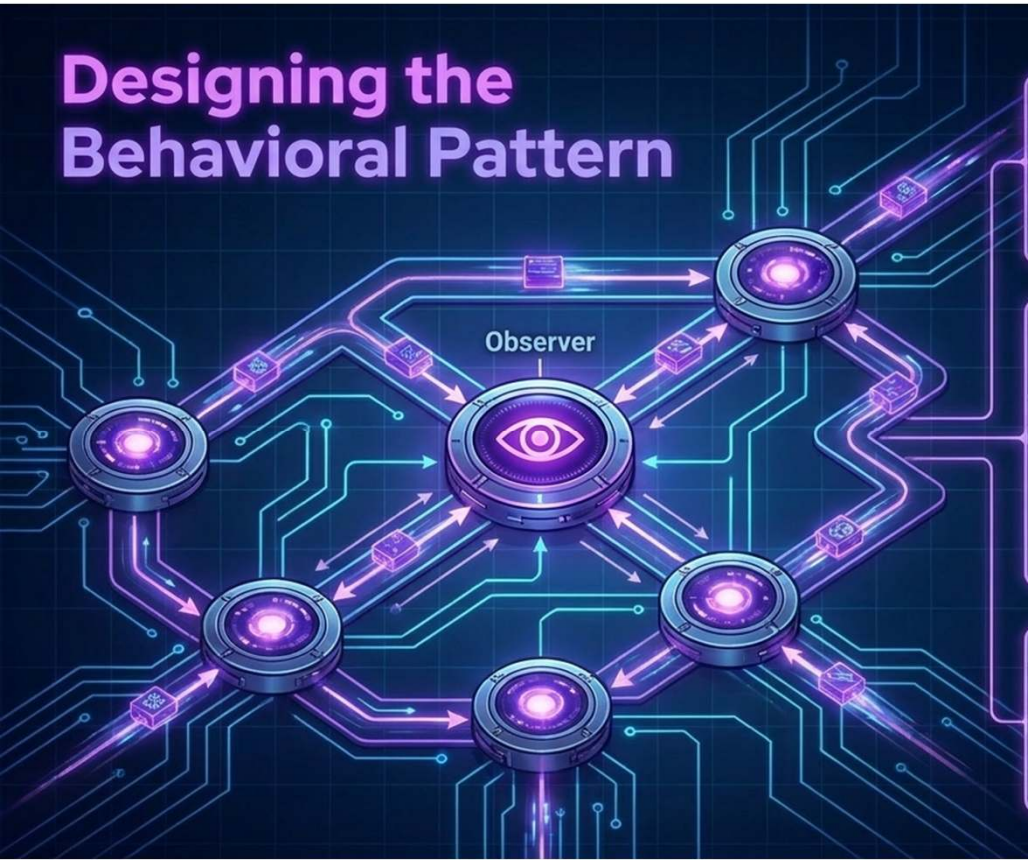
Designing the Structural Pattern



Core Principle:
Define how classes and objects are composed to form larger, efficient structures.

Examples:
Adapter, Bridge, Composite, Decorator, Facade.

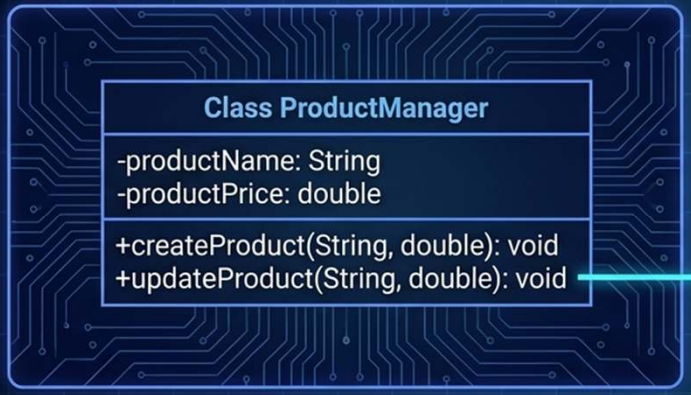
Designing the Behavioral Pattern



Core Principle:
Define the communication, assignment of responsibilities, and routing logic between objects.

Examples:
Observer, Strategy, Command, State.

From Architecture to Code: The Translation




```java
public class ProductManager {
    private String productName;
    private double productPrice;

    public void createProduct(String name,
        double price) {
        // Implementation
    }

    public void updateProduct(String name,
        double price) {
        // Implementation
    }
}
```

The Rule:
Code samples are not written in a vacuum. They must be direct structural translations of the finalized UML and Pattern diagrams.

The Execution:
If an attribute or method exists in your code, it must exist in your UML diagram . If it is in the UML, it must be accounted for in the implementation.

Standards for Presenting Code Samples

Highlighting:
Always use syntax highlighting for readability.

Focus:
Keep snippets strictly focused on the patterns and core logic. Omit unnecessary boilerplate or standard library imports.

Traceability:
Comment heavily. Map lines of code explicitly back to the sequence and UML architecture diagrams.

```
public class OrderProcessor {
    // Singleton instance pattern for global access
    private static OrderProcessor instance; // Singleton pattern: Ensures single instance.

    private OrderProcessor() {}

    // Factory Method for creating specific order handlers
    public OrderHandler createHandler(String orderType) {
        if ("DIGITAL".equals(orderType)) {
            return new DigitalOrderHandler(); // Concrete handler
        } else if ("PHYSICAL".equals(orderType)) {
            return new PhysicalOrderHandler(); // Concrete handler
        }
        return null;
    }
    // Lazy initialization for  Singleton: Created only when needed.
    public static synchronized OrderProcessor getInstance() {
        // Lazy initialization for Singleton
        if (instance == null) {
            instance = new OrderProcessor();
        }
        return instance;
    }
}
```